CUADERNILLOS DE DESARROLLO DE SOFTWARE

01

Arquitectura y Diseño de Interfaces Teoría

El mapa conceptual de la materia: del Vibe Coding al Spec-Driven Development, estilos arquitectónicos, C4, ADR, usabilidad, interfaces web y mobile, CI/CD.



Arquitectura y Diseño de Interfaces · 3° año · Ciclo 2026

Tecnicatura Superior en Desarrollo de Software · IES 9-018 · Prof. Paulo Alvarez

Fuente primaria: `github.com/IES9018/ADI-teoria-y-recursos`

Se lee junto con: `proyecto-adi-2026` · `proyecto-pp3-2026`

Índice

Cómo usar este cuadernillo

Del Vibe Coding al Spec-Driven Development

Arquitectura de Software

Diseño de Interfaces

Arquitectura Web

Arquitectura Mobile

CI/CD y Herramientas

Mapa conceptual final

Cómo usar este cuadernillo

> **Propósito:** que tengas el mapa conceptual de la materia antes de abrir cualquier archivo del repositorio. Este cuadernillo no reemplaza el repositorio de teoría: lo estructura, relaciona sus conceptos y te dice dónde profundizar.

El mapa de los tres repositorios

Repositorio	Qué contiene	Cuadernillo
`IES9018/ADI-teoria-y-recursos`	Teoría: conceptos, definiciones, ejemplos, analogías, fuentes	**Este** — Teoría
`IES9018/proyecto-adi-2026`	Consignas de TP, plantillas, proyecto modelo con código real	Cuadernillo 02 — Proyecto
`IES9018/proyecto-pp3-2026`	Reglas de los 3 sprints, checklists, rúbricas	Cuadernillo 03 — PP3

> RECORDÁ
> Estudiás la teoría acá; después aplicás los conceptos en los TP del repositorio de proyecto.

Cómo está organizado

Cada unidad del cuadernillo corresponde a una unidad del repositorio. Seguí el orden: los conceptos se construyen uno sobre otro.

Sistema visual de la colección

Cada bloque de color tiene un significado fijo. Si ves un recuadro y no entendés qué significa, es un error del cuadernillo: reportalo.

> CONCEPTO CLAVE
> **CONCEPTO CLAVE:** definiciones centrales de la materia.

> EN SIMPLE
> **EN SIMPLE:** explicación intuitiva o analogía cotidiana.

> ¿POR QUÉ IMPORTA?
> **¿POR QUÉ IMPORTA?** consecuencia práctica de saber o no saber esto.

ERROR FRECUENTE

ERROR FRECUENTE: equivocación común y cómo evitarla.

NO CONFUNDIR: dos conceptos que parecen iguales pero no lo son.

EN EL PROYECTO

EN EL PROYECTO: dónde aparece este concepto en el proyecto integrador.

CHECKPOINT — RESPONDÉ SIN MIRAR

CHECKPOINT: preguntas breves para verificar que entendiste antes de seguir.

Referencias a otras unidades

Cuando veas `VER EN EL REPO → ruta`, es material de profundización que no está reproducido en el cuadernillo. Consultalo cuando lo necesites.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuántos repositorios forman esta colección y qué rol cumple cada uno?
2. ¿Qué significa que un bloque tenga un color fijo en esta colección?

Del Vibe Coding al Spec-Driven Development

> La diferencia entre un experimento y un proyecto profesional es la especificación. Esta unidad te da el marco de trabajo para pasar de “probar código” a “construir con criterio”.

El problema: Vibe Coding

El Vibe Coding consiste en pedirle a una IA que genere código con descripciones vagas y aceptar el resultado sin auditoría. El sistema funciona hasta que crece, y entonces nadie sabe por qué hace lo que hace.

ERROR FRECUENTE

ERROR FRECUENTE: usar la IA para producir código sin haber definido primero qué debe hacer ese código. El resultado es deuda técnica invisible: funciona en el demo pero no escala, no se mantiene y no se audita.

La solución: Spec-Driven Development (SDD)

SDD invierte el orden: **primero especificás, después codificás**.

Etapa	Qué hacés	Artefacto
1. Especificar	Definís qué hace el sistema y qué NO hace	`SPEC.md`
2. Decidir	Documentás decisiones de arquitectura con alternativas	`docs/adr/ADR-00X.md`
3. Arnés	Configurás las reglas que tu agente IA debe respetar	`.opencoderules`
4. Generar	La IA produce código bajo esas restricciones	Código fuente
5. Auditar	Revisás línea por línea antes de hacer commit	Pull Request + checklist

CONCEPTO CLAVE

SPEC.md: archivo en la raíz del repo que describe el propósito, alcance, interfaz, reglas de dominio y Non-Goals del sistema. No es un documento de diseño: es el contrato contra el que se mide si el código es correcto.

Non-Goals

Los Non-Goals son tan importantes como los goals. Declarar explícitamente lo que NO hace esta versión evita que la IA (o el equipo) agregue funcionalidades no solicitadas.

> EN SIMPLE
>
> **EN SIMPLE:** si no escribís "esta versión no implementa OAuth2", la IA puede intentar agregarlo, gastar tokens y generar código que no pidieron.

El arnés (`.opencoderules`)

El arnés es un archivo de reglas en la raíz del repositorio que limita el comportamiento del agente IA. Define tres zonas:

Zona	Qué hace	Ejemplo
Alcance permitido	Qué tecnologías, patrones y estándares debe usar	"Usar TypeScript con tipado estricto"
Estándares obligatorios	Convenciones que no se negocian	"Conventional Commits, SPEC primero"
Prácticas prohibidas	Lo que la IA no debe hacer	"No usar `any`, no exponer secrets"

> EN EL PROYECTO
>
> **EN EL PROYECTO:** tu repositorio debe incluir `SPEC.md` y `.opencoderules`. La plantilla de SPEC está en `templates/SPEC-template.md` del repo `proyecto-adi-2026`. Los detalles de implementación por sprint se explican en el Cuadernillo 03.

Vibe Coding ≠ SDD. El primero delega el diseño en la IA; el segundo pone al estudiante como Arquitecto que especifica, decide, dirige y audita. La IA ejecuta, no decide.

> CHECKPOINT — RESPONDÉ SIN MIRAR
>
> **Respondé sin mirar:**
>
> 1. ¿Cuál es el orden correcto del flujo SDD?
> 2. ¿Qué es un Non-Goal y por qué es importante declararlo?
> 3. ¿Qué zona del arnés impide que la IA use una tecnología no autorizada?

Arquitectura de Software

> Arquitectar no es dibujar cajas y flechas: es tomar decisiones estructurales caras de revertir y registrarlas para que el equipo entienda por qué el sistema es como es.

Conceptos fundamentales

CONCEPTO CLAVE

Arquitectura de software: la estructura o estructuras del sistema, que comprenden los elementos del software, sus propiedades visibles externamente y las relaciones entre ellos (Bass, Clements, Kazman).

Término	Definición
Componente	Pieza con responsabilidad clara e interfaz (ej: módulo de autenticación)
Conector	Canal de comunicación entre componentes (ej: API REST, cola de mensajes)
Propiedad	Característica medible de un componente (rendimiento, seguridad, disponibilidad)
Vista arquitectónica	Lectura del sistema desde un ángulo: módulos, ejecución, despliegue

EN SIMPLE

EN SIMPLE: la arquitectura es el plano estructural del sistema. Cambiarla después de construir es carísimo, igual que mover una columna de carga en un edificio terminado.

VER EN EL REPO → `unidad-2-arquitectura-de-software/01-conceptos-generales/README.md`

Estilos arquitectónicos

Un estilo es una familia de decisiones repetibles sobre cómo se organizan los componentes.

Estilo	Cómo organiza	Cuándo elegirlo
Capas	Separación vertical estricta (presentación → lógica → datos)	Sistemas empresariales, MVPs
Tuberías y filtros	Componentes encadenados donde cada uno transforma entrada en salida	Compiladores, ETL, streams
MVC	Separa datos (Modelo), interfaz (Vista) y lógica de entrada (Controlador)	Aplicaciones con interfaz gráfica
Hexagonal	El dominio vive al centro; el exterior se conecta por puertos y adaptadores intercambiables	Sistemas donde el dominio es más valioso que la tecnología

EN EL PROYECTO

EN EL PROYECTO: el proyecto modelo del repositorio `proyecto-adi-2026` usa arquitectura hexagonal. Ver `03_ARQUITECTURA/` para el diagrama de contenedores y la justificación en los ADR.

Comparativa detallada: `unidad-2-arquitectura-de-software/02-estilos-arquitectonicos/README.md`

Modelo C4

El modelo C4 (Simon Brown) proporciona cuatro niveles de zoom para comunicar la arquitectura a distintas audiencias.

Nivel	Qué muestra	Para quién
C1 — Contexto	El sistema y su entorno: usuarios, sistemas externos	Todos los stakeholders
C2 — Contenedores	Aplicaciones, servicios, bases de datos y protocolos	Equipo de desarrollo
C3 — Componentes	Componentes internos de un contenedor	Desarrolladores del módulo
C4 — Código	Clases, interfaces, relaciones de implementación	Quien escribe código

Cada diagrama debe responder **una pregunta**. Si no podés escribir la pregunta al pie, el diagrama no cumple su función.

VER EN EL REPO → `unidad-2-arquitectura-de-software/04-vistas-y-escenarios/README.md`

ADR (Architecture Decision Record)

Un ADR documenta una decisión de arquitectura, su contexto, las alternativas consideradas y sus consecuencias. Es un registro inmutable: no se modifica, se crea uno nuevo si la decisión cambia.

Sección	Qué contiene
Título	ADR-NNN — título descriptivo
Contexto	¿Qué problema motiva esta decisión?
Decisión	¿Qué elegimos y por qué?
Alternativas	¿Qué otras opciones consideramos y por qué las descartamos?
Consecuencias	¿Qué cambia en el sistema con esta decisión?

Plantilla oficial: `templates/ADR-template.md` en el repo `proyecto-adi-2026`.

CONCEPTO CLAVE

Regla de oro: ninguna decisión técnica importante queda en la cabeza de una sola persona. Si no está documentada en un ADR, no pasó la auditoría.

Requisitos funcionales y no funcionales

Tipo	Define	Dónde vive
Funcional	Qué hace el sistema	`SPEC.md`, historias de usuario
No funcional (atributo de calidad)	Cómo lo hace: rendimiento, seguridad, disponibilidad	`SPEC.md` + ADR

La arquitectura está profundamente ligada a los atributos de calidad. Un requisito funcional puede cumplirse con muchas arquitecturas; un requisito no funcional casi siempre fuerza una estructura específica.

ERROR FRECUENTE

ERROR FRECUENTE: definir la arquitectura antes de conocer los requisitos no funcionales. Construir una API REST con microservicios cuando un monolito modular alcanza es sobrediseño. Construir un monolito cuando el sistema necesita escalar por servicio individual es deuda técnica.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuál es la diferencia entre un componente y un conector?
2. ¿Qué información mostrarías en un C1 que todavía no debería aparecer en un C2?
3. ¿Por qué un ADR no se modifica cuando la decisión cambia?

Diseño de Interfaces

> Una interfaz no se diseña para quedar bonita: se diseña para que una persona concreta logre su objetivo con la menor fricción posible.

HCI (Human-Computer Interaction)

La HCI estudia cómo las personas usan sistemas computacionales y cómo diseñar para minimizar error y esfuerzo. No es opinión: es evidencia acumulada durante décadas.

> CONCEPTO CLAVE
>
> **Usabilidad:** eficacia, eficiencia y satisfacción con las que un usuario logra sus objetivos (ISO 9241). Es medible, no decorativa.

UX ≠ UI. UX es la experiencia completa del recorrido (antes, durante y después del uso). UI es la capa visible e interactiva. Una interfaz linda puede sostener una experiencia pésima si el usuario no encuentra lo que busca.

Las 10 heurísticas de usabilidad (Jakob Nielsen)

Son principios de diseño, no opiniones. Se aplican antes de escribir código, durante el diseño de la interfaz.

#	Heurística	En criollo
1	Visibilidad del estado del sistema	El usuario siempre sabe qué está pasando
2	Correspondencia sistema-mundo real	Usá el lenguaje del usuario, no el del sistema
3	Control y libertad del usuario	“Deshacer” salva vidas
4	Consistencia y estándares	Misma palabra = misma acción siempre
5	Prevención de errores	Mejor que dar un buen mensaje de error
6	Reconocer antes que recordar	Mostrá opciones visibles, no obligues a recordar
7	Flexibilidad y eficiencia de uso	Atajos para expertos sin romper para novatos
8	Diseño estético y minimalista	Cada información extra compite con la relevante
9	Ayudar a reconocer y recuperarse de errores	Decí qué pasó y cómo solucionarlo
10	Ayuda y documentación	Idealmente no hace falta; si hace falta, que sea buscable

> VER EN EL REPO → `unidad-3-diseno-de-interfaces/01-hci-y-usabilidad/README.md`

Ergonomía de software

Adaptar la interfaz a las capacidades humanas no es opcional: es ingeniería.

Principio	Regla
Memoria de trabajo limitada	No obligues al usuario a recordar información de una pantalla a otra
Objetivos de toque	Área táctil mínima 44×44 px en mobile
Jerarquía visual	Lo más importante se ve primero (tamaño, color, posición)
Contraste	Relación 4.5:1 entre texto y fondo (WCAG 2.2 AA)

Métodos de diseño

De menor a mayor fidelidad, cada método responde una pregunta distinta.

CONCEPTO CLAVE

Wireframe: boceto de baja fidelidad que define estructura y jerarquía, sin color ni tipografía final. Se hace en papel o herramientas como Excalidraw.

Mockup: representación estática de alta fidelidad con colores, tipografía y componentes finales.

Prototipo: versión navegable que simula el flujo real para testear antes de programar.

Método	Pregunta que responde	Tiempo
Wireframe	¿Está todo lo necesario?	Minutos
Mockup	¿Se ve y se lee bien?	Horas
Prototipo	¿El recorrido funciona?	Horas–días

Sistema de diseño

Un sistema de diseño es una biblioteca de tokens (colores, espaciados, tipografía) + componentes reutilizables + reglas de uso. No es un archivo de Figma: es el contrato visual del equipo.

Composición mínima:

- **Tokens:** paleta con contraste validado, escala tipográfica, espaciados en múltiplos de 4/8 px
- **Componentes:** botones, inputs, tablas, cards con sus estados (default, hover, focus, disabled, error)
- **Reglas:** cuándo usar cada componente, qué no hacer

EN EL PROYECTO

EN EL PROYECTO: tu `SPEC.md` debe incluir los tokens del sistema de diseño. La especificación visual de tu interfaz es parte del contrato, no un archivo separado.

VER EN EL REPO → `unidad-3-diseno-de-interfaces/05-patrones-de-diseno/README.md`

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuál es la diferencia entre wireframe, mockup y prototipo?
2. ¿Por qué el tamaño mínimo de un botón táctil no es una decisión estética sino de ingeniería?
3. Nombrá tres heurísticas de Nielsen y explicá cada una en una frase.

Arquitectura Web

> La web moderna es un sistema distribuido donde la pregunta “¿dónde se arma la pantalla?” define rendimiento, SEO, costos de infraestructura y experiencia de usuario.

Modelo cliente-servidor

El cliente (navegador) solicita recursos; el servidor procesa y responde bajo el protocolo HTTP, que es inherentemente sin estado. Cada petición es independiente.

Estrategias de renderizado

Estrategia	¿Dónde se arma la página?	Primer render	Carga del servidor
CSR (Client-Side Rendering)	Navegador del usuario	Lento inicial	Baja
SSR (Server-Side Rendering)	Servidor en cada request	Rápido	Alta
SSG (Static Site Generation)	Servidor en build	Instantáneo	Casi nula
ISR (Incremental Static Regeneration)	Servidor en build + revalidación	Instantáneo para páginas cacheadas	Baja

EN SIMPLE

EN SIMPLE: CSR es comprar comida congelada y cocinarla en casa (viaja liviana, tu horno trabaja). SSR es comer en el salón (el chef prepara todo, el plato llega listo). SSG es el buffet preparado a la mañana (sale instantáneo, se regenera cada tanto).

REST y contratos de API

REST no es un protocolo: es un estilo arquitectónico donde los recursos se identifican por URL y se manipulan con verbos HTTP.

Verbo	Operación	Código HTTP esperado
`GET`	Leer recurso	200 OK
`POST`	Crear recurso	201 Created
`PUT` / `PATCH`	Actualizar recurso	200 OK / 204 No Content
`DELETE`	Eliminar recurso	204 No Content
	Error de validación	400 Bad Request
	Conflicto	409 Conflict

CONCEPTO CLAVE

OpenAPI: estándar para describir contratos REST en formato YAML o JSON. Herramientas como Swagger generan documentación interactiva y validan que la implementación cumpla el contrato. En la materia, el contrato se escribe primero en `SPEC.md` y se expone como `api-contracts.yaml` .

Stack de referencia

La materia utiliza: **Next.js** (SSR + CSR híbrido con server y client components) + **FastAPI** (Python, validación Pydantic, OpenAPI automático) + **PostgreSQL** (base relacional transaccional).

Componente	Tecnología	Responsabilidad
Frontend	React / Next.js	Interfaz de usuario, renderizado híbrido
Backend	FastAPI / NestJS	API REST, lógica de aplicación
Base de datos	PostgreSQL	Persistencia transaccional

VER EN EL REPO → `unidad-4-arquitectura-web/` para evolución web, diseño de sitios y patrones de diseño web.

EN EL PROYECTO

EN EL PROYECTO: tu API debe documentarse con OpenAPI. El contrato se define en `SPEC.md` antes de escribir código. El stack del proyecto modelo es Next.js + FastAPI + PostgreSQL.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Qué problema resuelve SSG que SSR no resuelve?
2. ¿Qué verbo HTTP usarías para crear un recurso y qué código esperar?
3. ¿Cuál es la diferencia entre un contrato REST definido en SPEC y uno generado automáticamente por el framework?

Arquitectura Mobile

> Mobile no es “la web achicada”. Es un entorno con batería finita, red intermitente, almacenamiento acotado y un sistema operativo que puede cerrar tu proceso cuando lo necesite.

Estrategias de desarrollo

Estrategia	Código	Performance	Acceso a hardware	Distribución
Nativa	Kotlin (Android), Swift (iOS)	Máxima	Completo	App stores
Cross-platform	Flutter, React Native	Muy buena	Amplio vía plugins	App stores
PWA (Progressive Web App)	HTML/CSS/JS	Limitada por navegador	Parcial	URL directa

EN SIMPLE

EN SIMPLE: nativo es construir con materiales locales y mano de obra especializada para cada plataforma. Cross-platform es una constructora única con planos que se adaptan a cada ciudad. PWA es lo que ya sabés hacer en la web, instalable en el dispositivo.

Offline-First

El patrón más importante del desarrollo mobile: la aplicación funciona plenamente sin conexión y sincroniza cuando hay red.

Componente	Función
Base local (SQLite)	Fuente primaria de datos: toda lectura/escritura va contra la base local
Repositorio	Capa única que abstrae si el origen es local o remoto
Cola de salida (outbox)	Cada cambio pendiente se marca y sincroniza cuando hay red
Sincronización bidireccional	El servidor y el cliente intercambian cambios con resolución de conflictos

CONCEPTO CLAVE

Offline-First: la red pasa a ser una optimización, no un requisito. La app es completamente funcional sin conectividad. El usuario nunca pierde datos por un error de red.

VER EN EL REPO → `unidad-5-arquitectura-mobile/` para conceptos mobile, patrones y herramientas de desarrollo.

EN EL PROYECTO

EN EL PROYECTO: la decisión de estrategia mobile (nativa, cross-platform o PWA) se documenta en un ADR con al menos dos alternativas descartadas. El TP5 del repo `proyecto-adi-2026` contiene la matriz de decisión. La implementación por sprint se detalla en el Cuadernillo 03.

ERROR FRECUENTE

ERROR FRECUENTE: diseñar la UI mobile como "la versión web pero más chica". Mobile tiene sus propios patrones de navegación, tiempos de carga, consumo de batería y estados de conectividad. Si no diseñás para la desconexión, tu app pierde datos cuando el usuario entra al subte.

CHECKPOINT — RESPONDÉ SIN MIRAR

Respondé sin mirar:

1. ¿Cuál es la ventaja principal de Offline-First sobre una app que solo funciona conectada?
2. ¿En qué caso elegirías una app nativa en lugar de una cross-platform?
3. ¿Qué componente evita que la UI sepa si los datos vienen de una base local o remota?

CI/CD y Herramientas

> Entregar software “a mano” es entregarlo con variabilidad: funciona en mi máquina, se rompe en producción y nadie sabe exactamente qué versión quedó publicada.

Contenedores (Docker)

Un contenedor empaqueta aplicación + dependencias + runtime en una imagen inmutable. La imagen se construye una vez y corre igual en cualquier entorno.

Concepto	Qué es
Imagen	Plantilla inmutable con todo lo necesario para correr la app
Contenedor	Instancia en ejecución de una imagen
Dockerfile	Receta para construir la imagen
Docker Compose	Orquestación de múltiples contenedores

EN SIMPLE

EN SIMPLE: “En mi máquina funciona” deja de ser excusa porque la máquina viaja dentro de la imagen.

Integración Continua (CI)

Cada push al repositorio dispara validaciones automáticas: linters, tests, builds. Romper la rama principal se detecta en minutos.

Componente del pipeline	Qué valida
Lint	El código cumple las reglas de estilo y sintaxis del lenguaje
Test	Las pruebas existentes siguen pasando
Build	La aplicación compila y empaqueta correctamente

Despliegue Continuo (CD)

Lo validado por CI llega a producción automáticamente (CD continuo) o a un estadio listo para publicar con un click (entrega continua).

GitHub Actions

Sistema de pipelines nativo de GitHub. Los workflows se escriben en YAML y se versionan junto al código.

Concepto	Descripción
Workflow	Archivo `.yml` que define el pipeline completo
Job	Conjunto de pasos que se ejecutan en el mismo runner
Step	Comando individual dentro de un job
Runner	Máquina virtual donde se ejecutan los jobs

> VER EN EL REPO → `unidad-6-herramientas-y-tecnologias/` para ejemplos de workflows YAML y automatización con arnés.

Automatización con arnés

El mismo principio de gobierno que aplicás a tu agente IA se aplica al pipeline: nada entra a `main` sin SPEC asociada, checks verdes y revisión.

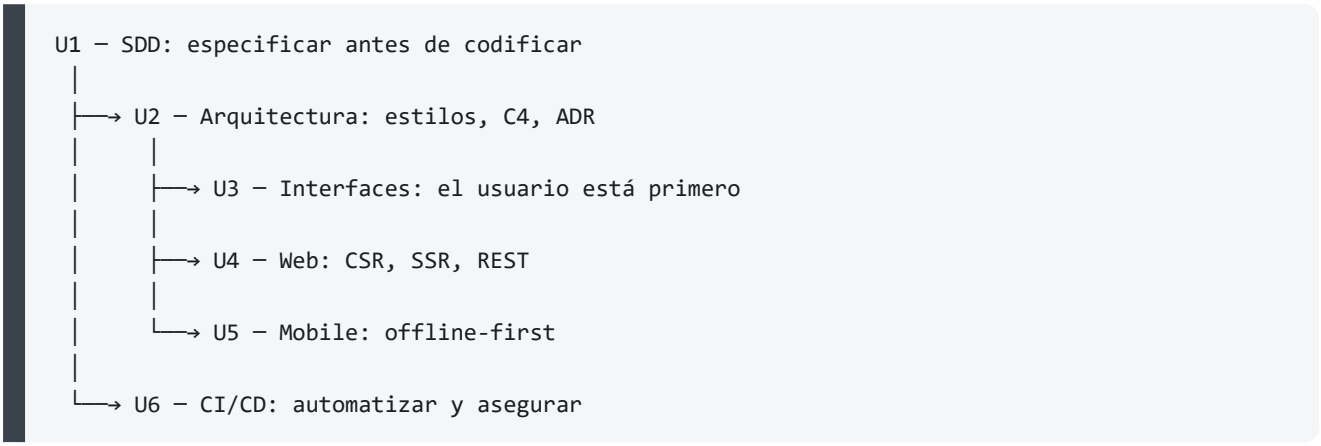
> EN EL PROYECTO
>
> **EN EL PROYECTO:** el pipeline CI/CD se documenta en un workflow de GitHub Actions que lintea, construye y valida SPEC en cada Pull Request. El proyecto modelo del repo `proyecto-adi-2026` contiene ejemplos. Los detalles de implementación y evaluación están en los Cuadernillos 02 y 03.

> Comparativa despliegue manual vs automatizado: `unidad-6-herramientas-y-tecnologias/06-integracion-y-prueba/README.md`

> CHECKPOINT — RESPONDÉ SIN MIRAR
>
> **Respondé sin mirar:**
>
> 1. ¿Qué problema resuelven los contenedores Docker?
> 2. ¿Cuál es la diferencia entre CI y CD?
> 3. ¿Qué archivos y en qué formato definen un pipeline en GitHub Actions?

Mapa conceptual final

Cada unidad de este cuadernillo te dio una pieza del rompecabezas. El orden importa porque los conceptos se construyen unos sobre otros.



¿POR QUÉ IMPORTA?

La conexión con los otros cuadernillos:

Cuadernillo	Qué hace con esta teoría
02 — Proyecto ADI	Aplica estos conceptos en 7 TP con el proyecto modelo
03 — PP3	Coordina los sprints donde implementás todo en tu repositorio individual

VER EN EL REPO → `RECURSOS.md` para la bibliografía completa y `recursos-generales/` para fuentes transversales.

RECORDÁ

El repositorio de teoría (`ADI-teoria-y-recursos`) tiene desarrollo completo de cada unidad, ejercicios y fuentes verificables. Este cuadernillo es el mapa; el repo es el territorio.

CHECKPOINT — RESPONDÉ SIN MIRAR

Revisión final:

1. Volvé a las unidades donde tengas dudas.
2. Revisá los checkpoints que no pudiste responder.
3. Cuando el docente habilite el TP, los conceptos de este cuadernillo son los que vas a aplicar.